

Discovering Integration Schemes for PDEs on Unstructured Domains via Reinforcement Learning

Extended Abstract

Ali Tayyar^{1*}, Joseph Bakarji¹

¹Mechanical Engineering, American University of Beirut, Riad El-Solh, Lebanon 1107 2020

Abstract

Designing integration schemes for partial differential equations (PDEs) requires balancing prediction accuracy against computational cost, with the optimal tradeoff depending on the structure of the equation. Adaptive techniques attempt to navigate this tradeoff through iterative algorithms, but typically rely on heuristics that do not always generalize across PDEs. We propose a reinforcement learning (RL) framework that casts next-point prediction in a spatio-temporal domain as a Markov Decision Process (MDP), enabling the discovery of efficient integration schemes without prescribed adaptive rules. Each integration step combines local linear (Taylor) approximations over a neighborhood of irregularly distributed space-time points, in the spirit of Generalized Finite Difference Methods (GFDM). The state consists of a set of walkers that converge on points where initial and boundary conditions are known, iteratively closing in on a target point of interest. The agent maximizes a terminal reward that balances prediction error against the number of integration steps. As a validation, the learned policy rediscovers the classical 2nd- and 4th-order finite difference stencils to machine precision. And it pushes the error-vs-cost Pareto front on the advection-diffusion and Burgers' equations.

Keywords: mesh-free methods, GFDM, reinforcement learning, PDE solvers, adaptive discretizations

1 Introduction

Classical numerical methods for PDEs, such as finite difference, finite element, finite volume often rely on pre-defined meshes whose structure critically determines both the accuracy and the computational cost of the solver. Adaptive Mesh Refinement (AMR) mitigates this by dynamically concentrating resolution where needed [7], and recent work has casted AMR as a reinforcement learning problem, with the agent learning to refine or coarsen cells based on solution features [3, 8]. These approaches, however, still operate within a fixed mesh structure and do not address whether the discretization itself can be learned from scratch.

Mesh-free methods remove the mesh entirely, operating on scattered point clouds and approximating differential operators through local stencils [5]. The Generalised Finite Difference Method (GFDM) constructs weights at each node by enforcing Taylor consistency constraints over a local neighbourhood [1, 4]. Moving Least Squares (MLS) [6] and RBF-based methods [2] offer complementary reconstruction strategies, though with their own challenges in stability and parameter tuning. In practice, point placement in all these methods is still determined by heuristic or user-defined strategies, and reconstruction accuracy is sensitive to the local point distribution.

This work adopts a different perspective: discretisation itself is treated as a sequential decision problem. Rather than fixing a grid, an agent learns where to evaluate the solution next in the space-time domain. The framework consists of two coupled components: (1) a reconstruction environment based on Taylor-constrained GFDM that computes the solution at any new point given n neighbours that were evaluated previously around the target point; and (2) an RL agent that learns a policy for selecting those points so as to maximise prediction accuracy while minimising total evaluations (computational cost).

*Corresponding author: amt35@mail.aub.edu.lb

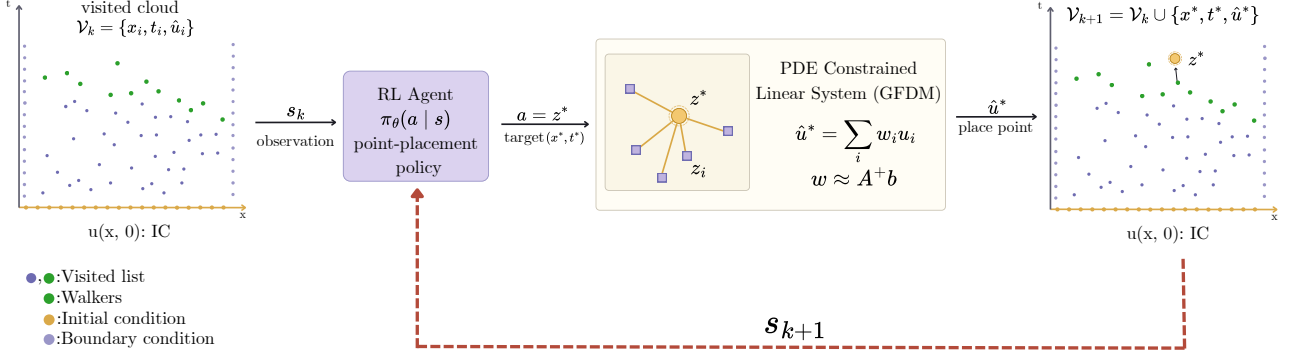


Figure 1: Framework mechanism. At each step k , the RL agent observes the current state s_k and outputs a proposed target point $a = z^*$; the GFD solver retrieves local neighbours from $\mathcal{V}_k$ and computes $\hat{u}^*$ via the PDE-constrained linear system; if accepted, the cloud is updated to $\mathcal{V}_{k+1}$ and the new state s_{k+1} is fed back to the agent. Green dots mark the current walker positions; blue-grey dots are previously visited points; orange and light blue dots mark the initial and boundary conditions respectively.

2 GFDM Reconstruction Framework

2.1 Local Linear Predictor

Given a target point $z^* = (x^*, t^*)$ and n neighbouring points $z_i = (x_i, t_i)$ with known solution values u_i , we seek a weighted combination

$$\hat{u}^* = \sum_{i=1}^n w_i u_i = \mathbf{w}^\top \mathbf{u}, \quad (1)$$

where the weights w_i depend only on the geometry of the neighbourhood through the offsets $\Delta x_i = x_i - x^*$, $\Delta t_i = t_i - t^*$.

2.2 Taylor Consistency Constraints

Expanding u_i in a Taylor series about z^* and substituting into (1), the prediction error vanishes to order p if and only if the weights satisfy the moment conditions

$$\sum_{i=1}^n w_i \Delta x_i^k \Delta t_i^\ell = \begin{cases} 1 & k = \ell = 0, \\ 0 & 1 \leq k + \ell \leq p. \end{cases} \quad (2)$$

In matrix form, $A\mathbf{w} = \mathbf{b}$, where $A \in \mathbb{R}^{m \times n}$ encodes the Taylor terms. For $m > n$ (more terms than neighbours) and thus an underdetermined system with a minimum-norm solution $\mathbf{w} = A^+ \mathbf{b}$ is computed via the pseudoinverse of A , giving the most stable weights.

2.3 PDE-Informed Constraint Matrix

So far, knowing the PDE does not affect the weights of the neighbours chosen. However, when u satisfies a known PDE, the partial derivatives in the Taylor expansion are not independent, and this linkage can be exploited to reduce the number of constraints while making the stencil physically informed.

Consider any PDE that is first-order in time and of order M in space:

$$u_t = \sum_{j=0}^M a_j(x, t) \frac{\partial^j u}{\partial x^j}. \quad (3)$$

Substituting (3) into the Taylor expansion of u_i about z^* eliminates u_t entirely, folding the temporal offset Δt_i into each spatial derivative order through the PDE coefficients. The contribution of the j -th spatial derivative

to the stencil is then governed by the single PDE constraint

$$c_{j,i} = \frac{1}{h^j} \left(\frac{\Delta x_i^j}{j!} + a_j \Delta t_i \right), \quad j = 0, 1, \dots, M, \quad (4)$$

where $h = \max_i |\Delta x_i|$. Each row of the Taylor system is replaced by the corresponding $c_{j,i}$.

The full constraint system combines the Taylor consistency rows with the PDE-informed rows into a single augmented system:

$$\begin{pmatrix} A \\ C \end{pmatrix} w = \begin{pmatrix} b \\ 0 \end{pmatrix}, \quad (5)$$

where $A \in \mathbb{R}^{m \times n}$ encodes the Taylor moment conditions, $b = (1, 0, \dots, 0)^\top$, and $C \in \mathbb{R}^{(M+1) \times n}$ encodes the PDE constraints with entries $c_{j,i}$ as defined above.

The framework extends to nonlinear PDEs by allowing the coefficients $a_j = a_j(u)$ in (3) to depend on the solution. Since $u(z^*)$ is unknown, each a_j is frozen at a local estimate computed from the neighbouring solution values:

$$\hat{a}_j = a_j \left(\frac{1}{n} \sum_{i=1}^n u_i \right), \quad (6)$$

and substituted into (4). This is a single Picard iteration: the nonlinear PDE is linearised locally at each target point using the available neighbourhood, and the constraint is assembled exactly as in the linear case.

The same substitution applies to any nonlinear PDE that can be written as $u_t = f(u) u_x + g(u) u_{xx} + \dots$ after linearisation: each nonlinear coefficient is frozen at its local neighbourhood estimate, and the constraint matrix is assembled exactly as in the linear case. This has been validated for Burgers' equation with the Hopf Cole analytical solution as a reference.

2.4 Error Bound

Under the Taylor constraints (2), the local reconstruction error is bounded by

$$|u^* - \hat{u}^*| \leq \|\mathbf{w}\|_1 \cdot C \cdot h^2, \quad (7)$$

where $C = \max(|u_{xx}|, |u_{xt}|, |u_{tt}|)$ at z^* and $\|\mathbf{w}\|_1 = \sum_i |w_i|$. The bound has two independent factors: the *geometric factor* Ch^2 improves as neighbours are placed closer to the target, while the *weight factor* $\|\mathbf{w}\|_1$ reflects stencil stability.

3 Space-Time Rollout and Validation

3.1 Algorithm

The reconstruction environment maintains a growing visited set of evaluated space-time points, initialised with the full initial condition. At each step, a candidate target point z^* is drawn; a KD-tree search is used to retrieve $n = 5$ neighbours where dx_i and dt_i are close enough to the visited set; the constraint system is assembled from the PDE coefficients and solved, and the point is accepted only if the stencil passes geometric and weight-admissibility checks (two-sided spatial coverage, $\|\mathbf{w}\|_1$ bounded, $\text{cond}(A) < 10^4$). Accepted points spawn new candidates in a forward box, propagating coverage through the domain.

3.2 Validation Results

We run the algorithm on the advection-diffusion and Burger's equations. The new points were chosen at random (will later be replaced by the RL agent), and the solution was compared against an analytical series solution. The method achieves mean error 3×10^{-4} over 30,000 accepted points (Fig. 2). As a consistency check, with $n = 3$ and $n = 5$ neighbours on a regular grid the method exactly recovers the classical and 4th FTCS weights, both to machine precision.

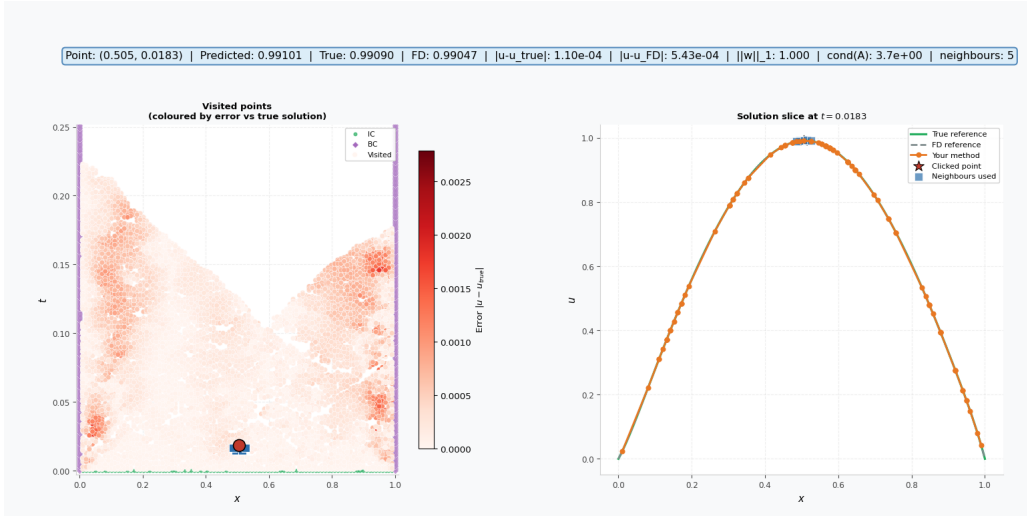


Figure 2: Left: space-time cloud of 30,000 visited points coloured by pointwise error $|u - u_{\text{true}}|$ for a diffusion advection system. The error is uniformly small across the domain ($\alpha = 0.05$, $c = 0.5$, $T = 0.5$). Right: solution slice at $t = 0.0183$.

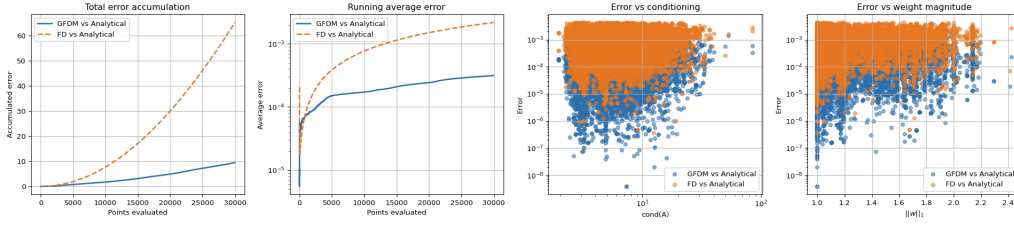


Figure 3: For a diffusion advection PDE, on the left: total accumulated error and running average vs. number of evaluated points (GFDM in blue, FD reference in orange). Right: per-point error vs. condition number of A and vs. $\|w\|_1$. GFDM clusters systematically below FD, consistent with the error bound (7).

4 Reinforcement Learning Formulation

4.1 Problem Setup and Mechanism

A set of N walkers $\{z_i^{(0)}\}_{i=1}^N$ is initialised at the initial condition. Separately, the algorithm maintains a growing visited list $\mathcal{V}^{(k)}$ of all evaluated space-time points including the IC, BCs, and all accepted GFDM solves, which serves as the data structure for the neighbour search. An episode terminates when any walker lands within tolerance ε of the target $z^* = (x^*, t^*)$, at which point the GFDM prediction at z^* is computed, and the terminal reward is issued.

4.2 Markov Decision Process

State. The state encodes the position and solution value of each walker, plus the global target:

$$s^{(k)} = [x_1^{(k)}, t_1^{(k)}, u_1^{(k)}, \dots, x_N^{(k)}, t_N^{(k)}, u_N^{(k)}, x^*, t^*] \in \mathbb{R}^{3N+2}. \quad (8)$$

The target (x^*, t^*) is appended once as global context. The visited list is entirely decoupled from the state: it lives in the background, grows unboundedly, and is used only for GFDM neighbour search. The state is updated only on accepted GFDM solves; rejected moves leave $s^{(k)}$ unchanged.

Action. A shared policy π_θ is applied independently to each walker. For walker i , the policy observes the local tuple $(x_i^{(k)}, t_i^{(k)}, u_i^{(k)}, x^*, t^*)$ and outputs a discrete displacement:

$$(\delta x_i, \delta t_i) \in \{-a_x, \dots, a_x\} \cdot \Delta x \times \{-a_t, \dots, a_t\} \cdot \Delta t, \quad (9)$$

with $a_x = a_t = 4$.

Reward. The reward is sparse zero at every step, with a single terminal signal:

$$r^{(K_{\text{term}})} = -|\hat{u}(z^*) - u_{\text{true}}(z^*)| - \lambda K_{\text{term}}, \quad (10)$$

where K_{term} counts all steps including rejected moves, $\hat{u}(z^*)$ is the GFDM prediction at z^* using $\mathcal{V}^{(K_{\text{term}})}$, and $\lambda > 0$ controls the accuracy/efficiency tradeoff.

5 Conclusions

We have presented a mesh-free PDE solver in which a validated GFDM reconstruction environment serves as the step function for an RL agent that learns where to place evaluation points in space-time. The reconstruction environment is fully operational, and exactly recovers classical FD stencils as special cases, and its dominant failure mode, geometrically poor stencils, is quantified by the derived error bound and will be addressed by the learned policy. The MDP formulation is designed so that any structure emerging in the learned trajectories reflects genuine discovery of PDE causal structure, not reward engineering.

References

- [1] J.J. Benito, F. Ureña, and L. Gavete. Influence of several factors in the generalised finite difference method. *Applied Mathematical Modelling*, 25(12):1039–1053, 2001.
- [2] B. Fornberg and N. Flyer. *A Primer on Radial Basis Functions with Applications to the Geosciences*. SIAM, Philadelphia, 2015.
- [3] C. Foucart, A. Charous, and P.F.J. Lermusiaux. Deep reinforcement learning for adaptive mesh refinement. *Journal of Computational Physics*, 491:112381, 2023.
- [4] L. Gavete, M.L. Gavete, and J.J. Benito. Improvements of generalised finite difference method and comparison with other meshless methods. *Applied Mathematical Modelling*, 27(10):831–847, 2003.
- [5] M. Griebel and M.A. Schweitzer. *Meshfree Methods for Partial Differential Equations II*. Springer, 2005.
- [6] P. Lancaster and K. Salkauskas. Surfaces generated by moving least squares methods. *Mathematics of Computation*, 37(155):141–158, 1981.
- [7] T. Plewa, T. Linde, and V.G. Weirs. *Adaptive Mesh Refinement – Theory and Applications*. Springer, 2005.
- [8] J. Yang et al. Reinforcement learning for adaptive mesh refinement. *arXiv preprint arXiv:2103.01342*, 2021.
- [9] Z. Zheng and X. Li. Theoretical analysis of the generalised finite difference method. *Computers & Mathematics with Applications*, 120:1–14, 2022.
- [10] A.M. Gambaruto. Local polynomial regression and filtering for a versatile mesh-free PDE solver. *International Journal for Numerical Methods in Fluids*, 2026. Early view, <https://doi.org/10.1002/flid.70069>.